

Figure 1: Three-layered monolithic application

executable or deployable unit. When deployed in a production environment, it runs as a single process. In Java terms, it is a WAR or EAR file, which is deployed on a Java EE container. These types of applications have modules, components and use one or more third party libraries. All such applications have three or more layers. How the layers commonly appear in a monolithic application can be seen in Figure 1.

The three layers are:

1. **Presentation layer:** This consists of a client side user interface mostly consisting of HTML, CSS, etc. It is also known as the UI layer.
 2. **Business layer:** This consists of business components, REST, etc.
 3. **Data layer:** This normally uses JPA, Hibernate, etc.
- Figure 2 illustrates the advantages of the monolithic application.

Issues related to the monolithic application

This approach works fine, but when the application becomes large and the team grows in size the drawbacks or issues start surfacing. The common issues with this type of approach are summed up below.

Difficult to deploy and maintain

If, for instance, you have an application up and running as shown in Figure 1, and you change some business components as in Figure 3, you will need to rebuild your whole application and replace the old archive ear/war (WAR is for Web Archives and EAR is Enterprise Archive in Java) with the new one. Although you haven't changed anything in the other layers, you will have to redeploy the whole application. A small change needs to be rigorously tested to ensure it doesn't affect any other part of the application. So if anything changes in the monolithic application, there is no way we can test and redeploy that part only—the whole application has to be redeployed. If the application is huge, then the size of the ear/war file is quite large and will take time in deploying. And deploying the whole application again will interrupt



Figure 2: Advantages of the monolithic application

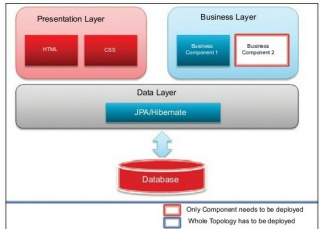


Figure 3: Changing the components in the business layer

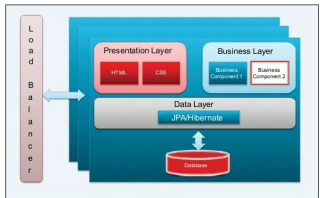


Figure 4: Scaling the application

the background tasks, irrespective of whether they are impacted by the change or not.

Stuck with the particular technology stack or framework

The other disadvantage is that we are stuck with the technology stack or framework for the long term. If any change is to be made in the framework or technology, the whole application will be affected and may need to be rewritten. For instance, if we have selected JVM, then we cannot think beyond it. Non-JVM languages and libraries cannot be used. Updating the JVM in the stack will lead to rigorous testing to ensure the application is working. If one component wants to use the latest feature